



SECURITY REVIEW • PREPARED FOR

StonkBrokers Smart LP v2

Uniswap v3 liquidity vaults on Robinhood
Chain

DATE

30.09.2026

CONDUCTED BY

Radev, Lead Security
Researcher

heavyw8t, Lead Security
Researcher

COMMIT

bfbf1b5f

REMEDICATION

fd7256bf

Phase & process

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Telegram [@sbsecurity_audits](https://t.me/sbsecurity_audits).

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 IMPACT

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2 LIKELIHOOD

- **High** - almost certain to happen, easy to perform, or highly incentivized.
- **Medium** - only conditionally possible, but still relatively likely.
- **Low** - requires specific state or little-to-no incentive.

02 EXECUTIVE SUMMARY

Key findings

TOTAL FINDINGS

21

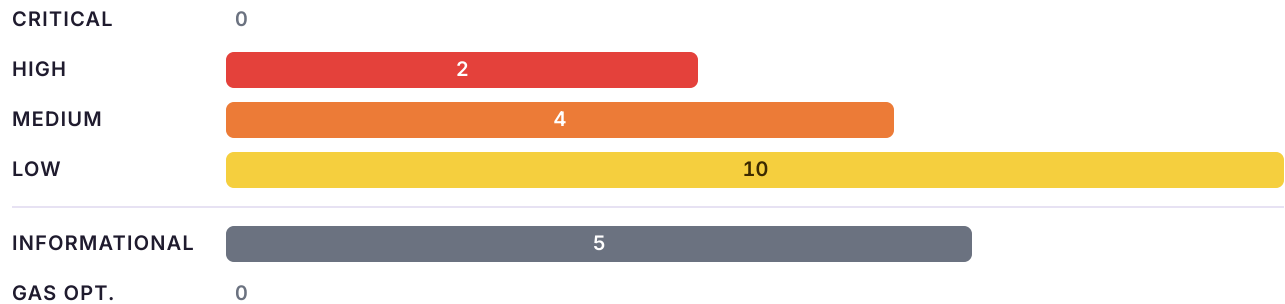
RESOLVED

21 of 21 · 100%

HIGH SEVERITY RESOLVED

2 of 2 · 100%

SEVERITY DISTRIBUTION

☒ Fixed & verified ☒ Acknowledged

CLIENT

**StonkBrokers
Smart LP v2**

REPOSITORY

Private

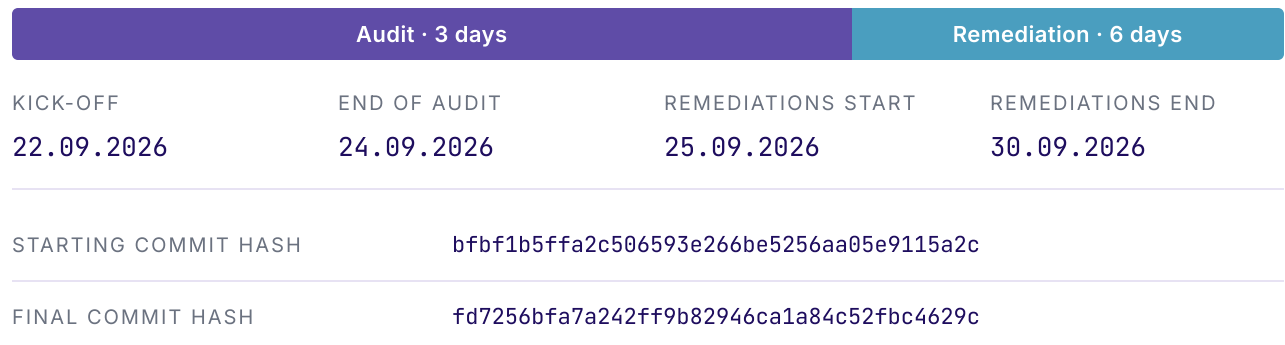
METHODS

**Manual review,
AI-Augmented**

CONTRACTS IN SCOPE

`contracts/smart-lp/SmartLpVault.sol`
`contracts/smart-lp/SmartLpFeeManager.sol`
(partial)`contracts/smart-lp/SmartLpLib.sol`

TIMELINE · OCTOBER 2026



03 PROJECT SUMMARY

What StonkBrokers Smart LP v2 is

StonkBrokers Smart LP v2 is a set of concentrated liquidity vaults on Robinhood Chain. Each vault is an ERC-20 share token over one Uniswap v3 position and runs in one fixed mode: full range, a balanced band that a keeper recenters under TWAP and price feed guards, or a one-sided ask that sells the base token as the price rises. Fees are either compounded into the position or converted by an optional fee manager into a chosen token that holders claim.

SECURITY POSTURE

BEFORE REMEDIATION

BFBF1B5F

Deposit pricing

H-01 • H-02 • L-09 • I-01

Deposits were priced at the pool TWAP while withdrawals paid out in kind at spot, so a one-sided deposit and a quick withdraw traded through the vault at a stale price. On vaults without a feed that gap could be created on purpose, a zap could end outside the band it had just checked, and the first deposit floor was skipped after a full exit.

Fee manager and harvest

M-01 • M-02 • L-07 • L-10 • I-02

Harvest mode credited fees to whoever held shares when harvest ran, not to the holders who earned them. A fee swap below the TWAP floor reverted the whole harvest, dust in one fee token blocked crediting of the other, the manager allowed a looser swap floor than the vault, and the Claimed event could name the wrong receiver.

Recenter and ask settlement

M-03 • M-04 • L-08

An ask vault's sale could unwind on a retrace because nothing settled the position. A band vault's recenter reverted when its rebalance swap missed the TWAP floor, and the swap cost fell only on holders who stayed through it.

AFTER REMEDIATION

FD7256BF

All 21 findings were fixed by the final commit.

Deposits into a funded vault are now priced on its own holdings at the pool's real price instead of the TWAP, which closes both Highs, and the zap's swap stops at the edge of the guard band. Fee transfers can no longer block withdrawals, recenters fill within the TWAP floor over several passes, ask vaults lock in their exit with a TWAP-gated settle, and harvests are sized by the caller and skip dust legs. The team turned each proof of concept into a regression test, and SB Security reviewed the fixes in two follow-up rounds.

BEFORE REMEDIATION

BFBF1B5F

AFTER REMEDIATION

FD7256BF

Withdraw and compound

L-01 • L-02 • L-03 • L-05 • L-06

A frozen fee recipient blocked every withdrawal. The permissionless compound added idle funds at an unchecked spot price and could lock a new ask vault before its first deposit, previewWithdraw did not match withdraw, and the share price read high during the native payout.

Oracle guards and ownership

L-04 • I-03 • I-04 • I-05

The vault did not check that the pool's observation ring covered the TWAP window, and three of four feed failure modes blocked deposits and the keeper instead of only widening the band. Ownership changed in a single step, and two comments on compound were out of date.

04 RECOMMENDATIONS

Beyond this audit

Measures that extend the protection of this review after launch, independent of any individual finding.

R-01

Keep the audit tests in CI

Run the regression, fork and economics suites on every change to the vault, library or fee manager, and treat any failure as a release blocker.

R-02

Monitor vaults on chain

Index vault and fee manager events and alert on pauses, failed keeper calls and wide gaps between spot and TWAP, so the team sees an unusual vault within minutes.

R-03

Keep admin keys under strong custody

Hold owner and guardian keys in hardware-backed multisigs, and tell holders about guard or fee changes before making them.

Core contracts

CONTRACT	ROLE
<code>SmartLpVault.sol</code>	The vault: an ERC-20 share token over one Uniswap v3 position, in full range, balanced band or ask mode. Handles deposits and zaps, in-kind withdrawals, compounding, keeper recents and the ask settlement.
<code>SmartLpLib.sol</code>	Linked library the vault delegatecalls for position and price math, deposit and withdraw amounts, guarded swaps and payouts. It keeps the vault under the contract size limit.
<code>SmartLpFeeManager.sol</code>	Optional per-vault fee engine. Converts the vault's collected fees into a fixed target token with TWAP-floored swaps, pays a crank reward and credits the rest to holders per share. In scope: the constructor wiring, the harvest call into compound and the checkpoint hook.

Privileged roles

Owner.

Raises the cap up to its fixed ceiling, tunes guards, band width and the early exit fee within fixed bounds, sets the deposit depth gate, names the keeper and guardian, and wires a fee manager once. It can also unpause deposits, sweep tokens that are not pool tokens, and hand over ownership in two steps.

Keeper.

Recenters balanced band vaults and trails ask vaults down, inside the TWAP and feed guards. The owner can also act as keeper.

Guardian.

Can pause deposits, recenters and trails. Only the owner can unpause, and withdrawals and compound are never paused.

Fee recipients.

Two fixed addresses, set at deployment, that split the performance fee taken from collected pool fees. They hold no other powers.

07 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
H-01	One-sided deposits exploit TWAP and spot price differences	HIGH	● FIXED
H-02	The TWAP can be manufactured on vaults with no feed, so the deposit price is chosen by the attacker	HIGH	● FIXED
M-01	Harvest mode credits fees to whoever holds shares at harvest time, not to whoever earned them	MEDIUM	● FIXED
M-02	Harvest reverts when the fee swap falls below the TWAP floor	MEDIUM	● FIXED
M-03	ASK sale unwinds on a retrace because nothing settles the position	MEDIUM	● FIXED
M-04	Recenter reverts when the rebalance swap misses the TWAP floor	MEDIUM	● FIXED
L-01	A frozen fee recipient blocks all withdrawals	LOW	● FIXED
L-02	<code>compound</code> adds idle funds at an unchecked spot price	LOW	● FIXED
L-03	<code>previewWithdraw</code> does not match <code>withdraw</code>	LOW	● FIXED
L-04	The vault does not verify that the pool's observation ring covers the TWAP window	LOW	● FIXED
L-05	<code>compound</code> can lock a new ASK vault before its first deposit	LOW	● FIXED

ID	TITLE	SEVERITY	STATUS
L-06	Share price reads too high during the native <code>withdraw</code> payout	LOW	● FIXED
L-07	A dust balance of one fee token blocks crediting of the other	LOW	● FIXED
L-08	Holders can step around the recenter swap cost by exiting first	LOW	● FIXED
L-09	<code>zapDeposit</code> can end outside the TWAP band it just enforced, closing every guarded path	LOW	● FIXED
L-10	Fee manager accepts a looser swap floor than the vault	LOW	● FIXED
I-01	First deposit floor is skipped after a full exit	INFORMATIONAL	● FIXED
I-02	The <code>claimed</code> event can emit <code>address(0)</code> as the fee receiver instead of the actual receiver	INFORMATIONAL	● FIXED
I-03	Stale comments on <code>compound()</code>	INFORMATIONAL	● FIXED
I-04	Missing two step ownership transfer	INFORMATIONAL	● FIXED
I-05	Three of four feed failure modes permanently block deposits and the keeper, against the documented guarantee	INFORMATIONAL	● FIXED

● HIGH SEVERITY · 2 FINDINGS

H-01**HIGH****● FIXED**

One-sided deposits exploit TWAP and spot price differences

`deposit()` values the deposit and the vault's holdings at the pool TWAP. `withdraw()` pays out a pro-rata share of the vault's tokens at spot, with no price conversion.

An attacker who deposits one token and withdraws in the same transaction gets back both tokens. This is a trade through the vault that costs only the 0.1% withdrawal fee. When spot has moved away from TWAP, the vault prices the trade at the stale TWAP, so the user sells above spot or buys below spot. Existing depositors pay the difference.

The attacker also does not need to move the price themselves. TWAP lags spot after any fast move, and `checkTwapBand()` allows gaps up to `maxSpotVsTwapBps` (10 to 500 ticks).

FULL_RANGE and BALANCED_BAND vaults are affected. ASK vaults are not, because `depositSingle()` accepts only the base token while the vault holds only the base.

IMPACT

Existing depositors lose principal each time a user trades through the vault after a fast price move. Common guard settings allow gaps of 1% to 4% between spot and TWAP, and the trade can be repeated after every move.

RECOMMENDATION

Require deposits to match the vault's redeemable basket. Mint shares as the minimum proportional contribution of either token, then refund excess amounts. Route one-sided deposits through `zapDeposit`, targeting the complete basket, including idle balances. Use token quantities for share accounting and TWAP only for safety and cap checks.

H-02

HIGH

● FIXED

The TWAP can be manufactured on vaults with no feed, so the deposit price is chosen by the attacker

Deposits are priced at the pool TWAP and `withdraw` pays out in kind, so a gap between the TWAP and spot is worth money to whoever deposits one token and leaves. On vaults that set `feed = 0` nothing cross-checks the TWAP and `SmartLpLib.sol#checkTwapBand()` only compares spot against that same TWAP.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLib.sol#L57-L91>

That leaves the attacker free to create the gap instead of waiting for one:

1. Push the pool price by a bit less than `maxSpotVsTwapBps`.
2. Hold it for `twapWindow` seconds so the TWAP follows.
3. Dump back to the starting price. Spot is now that far below a stale TWAP, still inside the guard.
4. Deposit the token the stale TWAP overvalues and withdraw in the same transaction.

The attack is cheapest when no feed is set and the TWAP window is short. With a 600 second window instead of 1800 and a 400 tick band, the attacker holds a price for ten minutes, not thirty, and is allowed a four times wider gap than the 100 tick default, while nothing can contradict the TWAP he just moved.

IMPACT

Holders lose value on every round and the attacker picks the timing. For example on a thin 1% pool with a 400 tick band and a 600 second window, a holder who deposits 36,000 exits with 35,759 at the same price, a loss of 240 (0.67% of the stake), while the attacker nets 162 after paying every swap fee. It repeats as often as the attacker likes. (this scenario is shown in the coded PoC below)

RECOMMENDATION

Price the deposit against the vault own basket instead of an absolute price, so a manipulated TWAP cannot move what a deposit is worth: mint on the minimum proportional contribution of both tokens and refund the rest. If that is too large a change, require a feed on any vault whose pool is thin and raise `twapWindow` well above the time an attacker can hold a price, since a 600 second window on a 1% pool is cheap to move.

● MEDIUM SEVERITY · 4 FINDINGS

M-01

MEDIUM

● FIXED

Harvest mode credits fees to whoever holds shares at harvest time, not to whoever earned them

In harvest mode the vault pushes net fees out to the fee manager on every deposit, withdraw, compound, recenter and trailDown - `SmartLpVault.sol#_collectFees()` runs from all of them. The manager then holds those tokens and credits nobody until `SmartLpFeeManager.sol#harvest()` runs. `accPerShareX128` is written in exactly one place, over the supply that exists at that moment.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpFeeManager.sol#L140>

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpFeeManager.sol#L170-L171>

`SmartLpVault.sol#_update()` calls `checkpoint` before any balance moves, so a new depositor's paid index is set to the current `accPerShareX128`. Everything already pushed but not yet harvested sits outside that index, so the new shares earn a full pro rata slice of it. `SmartLpVault.sol#deposit()` makes this explicit: `_collectFees()` runs before `_mintShares`, so those fees physically leave the vault before the new shares exist and are still credited to them.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L788-L792>

Three consequences follow from the same root cause:

- A depositor who arrives just before a harvest takes a pro rata slice of fees earned entirely before they arrived.
- A holder who withdraws before the next harvest loses the fees pushed during their own withdraw. `_collectFees()` runs at the top of `SmartLpVault.sol#_withdrawInKind()` and the burn happens at the bottom, so their own last push leaves the vault and then `checkpoint` settles them at an index that has not moved. Measured below: two holders earn the fees 50/50, the one who exits first receives 0 and the other takes the whole 4,789 USDG.
- When the last holder exits, whatever is still uncredited goes to the 1000 dead shares and is locked forever. Measured below: 4,789.40 USDG of a 4,837.78 USDG harvest is credited to `0x...dEaD`, the holder who earned it gets 0 and a later depositor does not inherit it.

The gap widens whenever a harvest cannot run. `harvest` converts the manager's whole balance through the pool against a TWAP-derived floor and reverts when the fill misses that floor. While it reverts nothing is credited, but fees keep arriving from the other push sites and `harvest` itself calls `compound()` before swapping. The uncredited pile grows for as long as the floor blocks the swap, then the first successful harvest credits all of it across whatever supply exists in that one block.

This is the opposite of what the `_update` NatSpec promises: "harvested fees always accrue to whoever held the shares while the fees were earned".

IMPACT

Fees are taken from the holders who earned them and handed to whoever holds shares when a harvest lands.

The round trip pays for itself once the uncredited pile exceeds `withdrawFeeBps` of post-deposit TVL, which is 0.1% at the deployed default of 10 bps. In the first test below a depositor puts about 1.08M into a 3.6M vault, claims 2,212 USDG of fees earned entirely before it arrived and nets +1,225 USDG after the withdraw fee, all in one block. The holder who owned the vault the whole time keeps 7,373 instead of 9,585.

A depositor who meant to hold anyway pays nothing at all - entering before a harvest is free.

The second test shows the amplified case. With harvest blocked by the swap floor ordinary vault activity grows the

M-01 · CONTINUED

uncredited pile to 20.1 NVDA + 3,681 USDG on a 3.6M vault. The depositor who front-runs the first working harvest then claims 4,805 USDG while the holder who was there the whole time is left with 2,402.

The loss is bounded by fees accrued since the last successful harvest. Harvest mode is not enabled on the live vaults today.

RECOMMENDATION

Credit the fees when they are pushed, not when they are converted. Have `collectSkimRoute` notify the manager with the net token0 and token1 amounts before any share change, keep a per share index for each of token0 and token1 and let `harvest` only convert and pay out against indexes that are already fixed. Then a deposit that lands after the fees were earned cannot dilute them and a stuck harvest only delays payment instead of changing who gets paid. Exclude the dead shares from the supply used for the index so a full exit does not strand the remainder.

M-02

MEDIUM

● FIXED

Harvest reverts when the fee swap falls below the TWAP floor

Vaults in harvest mode push their collected pool fees to `SmartLpFeeManager`, which converts them into the immutable target token. `harvest()` (`SmartLpFeeManager.sol:134-170`) reads `balanceOf(address(this))` for each non-target token and passes the whole balance to `SmartLpLib.swapWithTwapFloor` (lines 146-159). That helper (`SmartLpLib.sol:128-148`) sets `minOut` to the TWAP-implied output minus `tolBps`, swaps with no price limit, and reverts with `SwapFloor()` when the fill lands under the floor. The tolerance is fixed at construction as `swapFloorBps + pool.fee() / 100` (lines 99 and 122), which is 130 bps for a 0.3% pool at the default setting.

The whole balance is always sold, and the contract exposes no amount or percentage parameter, no owner, and no sweep. Four situations push the fill under the floor:

- Liquidity in the pool falls between fee accrual and the crank, so the same size moves the price past the tolerance, or in the case of a route pool being used, that pool has low liquidity relative to the fee amount in general.
- Nobody cranks for a long period, so the accrued balance grows past the size the pool can absorb.
- Spot offers a worse price than the TWAP-implied quote in the direction of the swap, by more than the tolerance. This reverts at any size, in either swap direction.
- An attacker transfers the token to be swapped directly to the fee manager. `harvest()` reads the raw balance, so the donation enlarges the swap until it breaks the floor.

IMPACT

The unconverted balances and all fees earned afterwards stay frozen for as long as the condition holds. A failed `harvest()` does not enlarge the manager's balance, because the revert also undoes the `compound()` it called at line 138. A separate `vault.compound()` call, which anyone can make, does push newly earned position fees into the stuck manager, and those fees cannot be harvested either.

`SmartLpFeeManager` is immutable and `setFeeManager` is set-once (`SmartLpVault.sol:778-783`), so a wired vault cannot be repaired.

RECOMMENDATION

We recommend that `harvest()` accept an input amount or a percentage of the balance, so a cranker can size each swap to the liquidity the pool currently offers. We further recommend tracking pushed fees in a state variable rather than reading `balanceOf`, so a transfer to the fee manager cannot enlarge the swap.

M-03

MEDIUM

● FIXED

ASK sale unwinds on a retrace because nothing settles the position

The rule the code enforces is narrower than the one the docs promise. The vault never deploys quote: `_deployIdle` zeroes the quote side in ASK mode, `compound` only folds idle base back in and `trailDown` only moves base.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L373-L382>

Nothing covers the position itself. The ASK band is a plain Uniswap v3 range order, so once price walks through it the quote sits inside the position, not idle in the vault. No function pulls it out at settlement, so a retrace converts it straight back to base.

`isSettled()` and `isAllBase()` are spot tick reads with no sticky flag, so after a full retrace the vault reads as open again.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L249-L253>

`depositSingle` reopens and `trailDown` re-arms the sell program, which contradicts its own comment "partially converted or settled: never re-arm".

The docs state the stronger rule: the NatSpec says ASK "settles to quote", `isSettled()` says the vault "holds quote only" and the Hashlock report lists "SINGLE SIDED NEVER ROUND-TRIPS - cannot silently buy back in" as an invariant.

IMPACT

Funds are never at risk, since `withdraw` is unconditional and every holder is on the same side. The cost is economic: a holder who sees "settled" and does not withdraw can end up back in base after a retrace, so the sell program silently unwinds. In the PoC a 1,000 NVDA vault settles at 193,280 USDG and, after price returns past the entry, holds 1,000 NVDA again while the holder exits with 175,132 USDG of value.

RECOMMENDATION

Add a permissionless `settle()` that collects fees, decreases the whole position to idle quote and burns the NFT, plus a sticky `settled` flag. Gate it on the TWAP as well as spot, so a one block push cannot settle the vault early. Mind what burning the NFT does: `positionId = 0` makes `isAllBase()` return true and `isSettled()` return false.

`depositSingle` then passes both of its gates and `_deployIdle` mints a fresh band out of idle base fees, after which `trailDown` works again. So check the sticky flag in `depositSingle` and in `_deployIdle`. `trailDown` needs no extra check while there is no position, since `NoPosition` already blocks it.

M-04

MEDIUM

● FIXED

Recenter reverts when the rebalance swap misses the TWAP floor

`recenter()` (`SmartLpVault.sol:646-670`) is supposed to be called regularly by the keeper when a `BALANCED_BAND` position becomes too distant from the current TWAP. The new position after `recenter()` spans both sides of the TWAP equally, so depending on how far the current TWAP sits from the previous position's center, part of the position's value, up to half, has to be converted into the other pool token.

To do that, `recenter()` uses `swapWithTwapFloor` (`SmartLpLib.sol:128-148`), which sends the whole amount in one exact-in call with no price limit and reverts `SwapFloor()` unless the full input clears the TWAP quote minus `tolBps` . That tolerance is `swapFloorBps` (100) plus the pool fee (30 bps at 0.3%), and the pool takes its fee out of the input, so only 100 bps is left to cover price impact.

Once a vault holds a large chunk of a pool's liquidity, any `recenter` that needs a real swap moves the price past those 100 bps and reverts, stopping the keeper from bringing the position back into the fee-earning range around the TWAP.

IMPACT

The vault stays out of range and earns no fees until it shrinks. The denial of service is temporary. Enough users withdrawing leaves a smaller amount to convert, and the keeper can also provide liquidity to the pool itself before calling `recenter()` .

RECOMMENDATION

We recommend allowing the keeper to `recenter` in parts, so the conversion is split into swaps the pool can absorb inside the floor, rather than loosening the slippage limit.

● LOW SEVERITY · 10 FINDINGS

L-01

LOW

● FIXED

A frozen fee recipient blocks all withdrawals

Every `withdraw` collects fees first. `SmartLpLib.sol#collectSkimRoute()` pushes the performance fee to `feeRecipientA` and `feeRecipientB` and, in harvest mode, the net fees to `feeManager`. All three are immutable and every transfer uses `safeTransfer`.

[https://github.com/SB-Security/audit-2026-09-](https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLib.sol#L249-L264)

[SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLib.sol#L249-L264](https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLib.sol#L249-L264)

USDG has a freeze list (`isFrozen`) and the stock tokens have a block list (`isBlocked` on the beacon). If one of these addresses gets frozen, the transfer reverts and `withdraw` reverts with it. The fees stay uncollected, so every retry reverts again.

IMPACT

All principal is stuck for every holder until the issuer lifts the freeze, with no admin fix. This breaks the "withdraw is ALWAYS live" guarantee from the docs.

RECOMMENDATION

Do not let a fee transfer revert the collection. On failure the amount stays idle in the vault and goes to the holders, which is safe because vault accounting is balance based.

```
function _trySend(address token, address to, uint256 amount) private {
    if (amount == 0) return;
    (bool ok, ) = token.call(abi.encodeCall(IERC20.transfer, (to, amount)));
    ok; // on failure the amount stays in the vault
}
```

L-02

LOW

● FIXED

compound adds idle funds at an unchecked spot price

`SmartLpVault.sol#compound()` is permissionless and adds all idle balances to the position. Unlike the deposit paths it never calls `_checkTwapBand()`. The liquidity is sized at spot in `SmartLpLib.sol#previewLiq()` and added with `amount0Min = amount1Min = 0`.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L638-L642>

In a band vault idle balances are not only fee dust. `deposit` accepts unmatched ratios that stay idle and pair deposits into an out of range band leave one side idle. An attacker pushes spot past the band edge, calls `compound` so the idle side enters the band and then swaps back through it at a discount. `harvest` also calls `compound`, so it is a second entry point.

IMPACT

Holders lose part of the idle balance each time. On a thin pool the vault lost about 2% of TVL in one transaction. On deeper pools or with small idle balances it is not profitable, which is why the likelihood is low.

RECOMMENDATION

Run the TWAP guard before redeploying idle funds, but skip the deploy instead of reverting, so fee collection and `harvest` keep working.

```
function compound() external nonReentrant returns (uint256 added0, uint256 added1) {
    _collectFees();
    - (added0, added1) = _deployIdle();
    + if (_twapBandOk()) (added0, added1) = _deployIdle(); // non-reverting version of
    _checkTwapBand
    emit Compounded(added0, added1);
}
```

L-03

LOW

● FIXED

previewWithdraw does not match withdraw

`SmartLpLens.sol#previewWithdraw()` is meant to tell an integrator what a holder would receive from `withdraw`. It diverges from the real payout in three separate ways, in both directions.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLens.sol#L93-L113>

```
(, int24 spot, , , , ) = vault.pool().slot0();
(uint256 p0, uint256 p1) = vault.positionAmountsAt(TickMath.getSqrtRatioAtTick(spot));
uint256 idle0 = vault.token0().balanceOf(vaultAddr);
uint256 idle1 = vault.token1().balanceOf(vaultAddr);
out0 = ((p0 + idle0) * shares) / ts;
```

1. It prices the position at the tick boundary. `getSqrtRatioAtTick(spot)` snaps to the bottom of the current tick, while `withdraw` settles against the pool's real `sqrPriceX96`. This makes the preview read slightly HIGH on one token.
2. The NatSpec says "before withdraw fee" but the code subtracts it. The returned value is already net, so an integrator who reads the comment literally and deducts `withdrawFeeBps` again understates the payout twice over.
3. It omits uncollected position fees, which makes it read LOW. `positionAmountsAt` is documented as principal only:

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L215-L216>

The real exit collects first. `_withdrawInKind` calls `_collectFees()` at the top and only then reads the idle balances it splits:

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L554-L573>

```
uint256 ts = totalSupply();
_collectFees();

uint256 idle0 = token0.balanceOf(address(this));
uint256 idle1 = token1.balanceOf(address(this));
```

In compound mode, which is what every live vault runs today, `feeManager` is zero, so

`SmartLpLib.sol#collectSkimRoute()` transfers nothing out after the performance-fee skim and the net collection simply stays in the vault as idle:

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLib.sol#L261-L264>

```
if (feeManager != address(0)) {
    if (c0 - s0 > 0) SafeERC20.safeTransfer(IERC20(token0), feeManager, c0 - s0);
    if (c1 - s1 > 0) SafeERC20.safeTransfer(IERC20(token1), feeManager, c1 - s1);
}
```

That freshly collected amount is then split pro rata to the withdrawer. The preview never counted it. The gap grows with trading volume and with the time since the vault was last touched by a deposit, withdraw or compound.

IMPACT

Nobody loses funds. The divergence is a correctness problem for anything that consumes the number.

Measured below on a FULL_RANGE vault after eight round-trip swaps with no vault interaction in between, the real payout

L-03 · CONTINUED

exceeds the preview by 4 bps on both tokens. Mechanism 1 pushes the other way, so on a given call the two partly cancel and the net sign depends on how much fee has accrued since the last touch.

The direction that actually breaks something is mechanism 1: a UI that feeds the preview straight into `min0` or `min1` gets `MinOut` reverts when the preview reads high. Mechanism 3 is the safe direction, since the user receives more than they were shown, but it still means any integrator computing slippage bounds, displayed proceeds or a tax basis from this number is working from a figure that does not match the chain.

RECOMMENDATION

Make the preview mirror what `_withdrawInKind` actually does. Use the real `sqrtPriceX96`, add the position's uncollected fees to the principal, then fix the comment so it matches the net value the function returns.

```
- (, int24 spot, , , , ) = vault.pool().slot0();
- (uint256 p0, uint256 p1) = vault.positionAmountsAt(TickMath.getSqrtRatioAtTick(spot));
+ (uint160 sqrtP, , , , , ) = vault.pool().slot0();
+ (uint256 p0, uint256 p1) = vault.positionAmountsAt(sqrtP);
```

For the fees, read `tokens0wed0` and `tokens0wed1` from the position through the NFPM together with the fee growth owed since the last collect, apply `perfFeeBps` the same way `collectSkimRoute` does, then add the remainder to `idle0` and `idle1` before the pro rata split. Where an exact mirror is impractical, documenting that the return value is a lower bound in compound mode would at least let integrators size their tolerance correctly.

L-04

LOW

● FIXED

The vault does not verify that the pool's observation ring covers the TWAP window

`SmartLpLib.twapTick` (`SmartLpLib.sol:41-51`) calls `pool.observe([1800, 0])` with no try/catch. If the pool holds no observation at least 1800 seconds old, `observe` reverts with `OLD` .

Currently, nothing guarantees that the pool's observation cardinality is large enough. The constructor (`SmartLpVault.sol:151-197`) takes any pool and pins `twapWindow` to 1800 without reading the ring. `setGuards` (line 733) and `SmartLpFeeManager` (line 87) do the same. A pool starts at cardinality 1, and only a manual `increaseObservationCardinalityNext` raises it: `ISmartLp.sol:34` declares that function, and no contract calls it. Once the ring wraps, each write overwrites the oldest entry, so the more the pool trades, the less history it keeps.

IMPACT

`deposit` , `zapDeposit` , `depositSingle` , `zapWithdraw` , `recenter` , and `trailDown` revert, as do the `twapTick()` and `totalValueQuote()` views. In-kind `withdraw` skips the guard, so depositors can still exit and no funds are locked.

RECOMMENDATION

We recommend that the constructor calls `observe([twapWindow, 0])` to check that old enough observations are available at launch, as well as growing the `observationCardinality` to at least 1800 before launch, to ensure `pool.observe([1800, 0])` will always function. The cost of this on the Robinhood chain is realistically in the one-digit dollar range at current gas prices.

L-05

LOW

● FIXED

compound can lock a new ASK vault before its first deposit

On an ASK vault with no position yet, `compound()` mints the first band itself. `SmartLpVault.sol#_deployIdle()` places it next to `spotTick()` and `compound()` runs no price guard.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L378-L382>

Right after a vault is deployed, an attacker pushes the base price down, sends a dust amount of base to the vault, calls `compound()` and swaps back. The band now sits below the real price, so `isSettled()` is true and the vault is 100% quote.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L520-L521>

`depositSingle` reverts with `DepositsClosed` and `trailDown` does too, because it needs an all base vault. Nobody can fix it: the owner has no function to remove the position and a depositor cannot push the price back, because `depositSingle` checks the TWAP.

IMPACT

The new ASK vault can never take a deposit and has to be redeployed. The attacker only pays pool fees on the round trip and can repeat it on every redeploy. And all live ASK vaults already have a position.

RECOMMENDATION

Let only `depositSingle`, which runs the TWAP guard, mint the first ASK band.

```
if (positionId == 0) {  
+   if (msg.sig == this.compound.selector) return (0, 0); // first band only from  
    depositSingle  
    (int24 lo, int24 hi) = _askTicks(spotTick());
```

L-06

LOW

● FIXED

Share price reads too high during the native withdraw payout

`withdraw` burns the shares first and then pays out. With `unwrapNative = true`, `SmartLpVault.sol#_payOut()` sends ETH to `to` before it transfers the ERC-20 leg.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L618-L631>

While `to` runs its `receive()`, the vault still holds the other leg that is about to be paid out, but `totalSupply` has already dropped. The integration guide tells integrators to price shares as `totalValueQuote() / totalSupply()` and that price is wrong inside this callback. The vault's own entry points are `nonReentrant`, so nothing in scope can be abused, but any view reader can.

IMPACT

A protocol that prices vault shares on chain (a lending market or an aggregator) can be fooled by a large withdrawer. Important to note is that if any lending, collateral or aggregator integration is planned, the impact becomes high but for now no on chain consumer exists yet.

RECOMMENDATION

Pay the ERC-20 legs before sending ETH, so the vault holds no pending payout while `to` runs.

```
function _payOut(uint256 out0, uint256 out1, bool unwrapNative, address to) private {
+   uint256 wOut;
    if (unwrapNative && weth != address(0)) {
-       uint256 wOut = weth == address(token0) ? out0 : out1;
-       if (wOut > 0) {
-           IWeth9Minimal(weth).withdraw(wOut);
-           (bool ok, ) = to.call{value: wOut}("");
-           if (!ok) revert EthSendFailed();
-           if (weth == address(token0)) out0 = 0;
-           else out1 = 0;
-       }
+       wOut = weth == address(token0) ? out0 : out1;
+       if (weth == address(token0)) out0 = 0;
+       else out1 = 0;
    }
    if (out0 > 0) token0.safeTransfer(to, out0);
    if (out1 > 0) token1.safeTransfer(to, out1);
+   if (wOut > 0) {
+       IWeth9Minimal(weth).withdraw(wOut);
+       (bool ok, ) = to.call{value: wOut}("");
+       if (!ok) revert EthSendFailed();
+   }
}
```

L-07

LOW

● FIXED

A dust balance of one fee token blocks crediting of the other

A dust amount of the non-target token in the fee manager causes `harvest()` to revert, so fees already held in the target token remain uncredited.

`harvest()` swaps every non-target balance to the target before it reads the target balance and accrues it (`SmartLpFeeManager.sol:145-172`). A revert in any swap therefore skips the accrual.

The swap reverts on dust. `SmartLpLib.swapWithTwapFloor` sets `minOut = implied - FullMath.mulDiv(implied, toLBps, 10_000)`. Being a percentage, the tolerance floors to zero when `implied < 10_000 / toLBps`, which is 76 output units at `toLBps = 130` and 909 at the minimum `toLBps = 11`.

`minOut` then equals the fee-free TWAP output; since the pool itself will ceiling-round the fee, this will lead to the returned value being smaller than `minOut`, and the leg reverts `SwapFloor()`.

Uniswap charges fees in the input token, so one-directional flow leaves dust of the other. Any address can also donate dust, since `harvest()` reads `balanceOf`.

IMPACT

Both the potentially relevant sized target token balance and the dust balance remain unharvested until an additional small amount of the non-target token is added, either manually or through the normal fee collection. This can especially be an issue on one-sided ask positions or very low activity or highly skewed pools.

RECOMMENDATION

Skip a conversion leg whose implied output falls below a minimum, so the dust stays in the manager for a later crank. Continue to the accrual and credit the target tokens already held.

L-08

LOW

● FIXED

Holders can step around the recenter swap cost by exiting first

`recenter()` (`SmartLpVault.sol:646-670`) swaps the vault's position toward the new TWAP band, and that cost lands on every share. Nothing blocks a user from exiting before a recenter. `lastRecenterAt` , `recenterCooldown` and the TWAP band bounds are public, so users will be able to predict the timeframe of the recenter. The user can withdraw in kind first before the recenter, only paying the 10bps fee, then re-deposit after the recenter.

A recenter converts up to half the vault, so the holder avoids half the pool fee, plus price impact:

- On a 0.05% pool, the recenter costs 2.5 bps, and the holder loses 7.5 bps by exiting.
- On a 0.3% pool, it costs 15 bps, and the holder gains 5 bps.
- On a 1% pool, it costs 50 bps, and the holder gains 40 bps.

IMPACT

Depending on the fee tier of the pool, this allows users to evade part of the recenter cost, while the remaining depositors pay the full cost.

RECOMMENDATION

We recommend setting `withdrawFeeBps` at or above half the pool's fee, which the 100 bps cap allows for every tier.

L-09

LOW

● FIXED

zapDeposit can end outside the TWAP band it just enforced, closing every guarded path

zapDeposit reads the TWAP band once at the top, then performs its own swap and never looks again.

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L477-L511>

```
int24 twap_ = _checkTwapBand();
...
(int24 zlo, int24 zhi) = _targetTicks(twap_);
(uint256 d0, uint256 d1) = SmartLpLib.zapSwap(address(pool), zlo, zhi, inIs0, rIn, twap_,
    _tolBps());
```

The swap that follows is floored by SmartLpLib.sol#swapWithTwapFloor(), which passes MIN_SQRT_RATIO + 1 / MAX_SQRT_RATIO - 1 as the price limit, so nothing bounds where the price ENDS UP:

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLib.sol#L134-L148>

```
uint256 implied = twapQuoteOut(zeroForOne, amountIn, twap_);
uint256 minOut = implied - FullMath.mulDiv(implied, tolBps, 10_000);
```

minOut constrains the AVERAGE fill price, not the final one. A swap whose average price sits just inside tolBps finishes at roughly twice that distance from the TWAP, so the transaction ends with spot well outside the band that _checkTwapBand() had just enforced. Two things follow in the same block:

1. Every guarded entry point now reverts TwapDeviation for everyone - deposit, zapDeposit, depositSingle, zapWithdraw and the keeper's recenter and trailDown. withdraw is unguarded so exits stay live.
2. _deployIdle() runs at the end of zapDeposit and commits the vault's whole idle balance to the position at that displaced price, which is a price the guard at the top of the same function would have refused to start from.

No manipulation is involved. An ordinary deposit, well inside the vault's own cap, does it.

IMPACT

Take a BALANCED_BAND vault with a 100 tick band and swapFloorBps 300 on a 0.3% pool (tolBps 330), whose pool holds about 54 WETH against a capQuote of 150 WETH. A single 20 WETH zapDeposit moves the spot-TWAP gap from 13 ticks to 179 and immediately locks out every guarded path, including the depositor's own zapWithdraw on the shares he just received.

The honest limit on this: it is temporary and it heals on its own. Nothing has to trade - the TWAP simply catches up to spot. In the same setup, the gap falls to 87 ticks after 5 minutes and to 0 after 10, at which point deposits work again. A vault on the 1800 second window would stay shut proportionally longer. No funds are ever at risk and withdraw is never blocked.

So this is a correctness problem with the guard rather than a way to take anything: the vault swaps at a price state it defines as unsafe and then builds its position on top of it.

RECOMMENDATION

Bound where the swap can end, not just its average price. Derive a sqrtPriceLimitX96 from twap_ and tolBps and pass it to pool.swap so the fill stops at the edge of the band instead of running through it and accept the resulting partial fill. Re-asserting _checkTwapBand() after the swap would also catch it, though it turns the case into a revert for the depositor rather than a smaller fill.

The same price-limit change is what SmartLpLib.sol#swapWithTwapFloor() needs for the recenter problem in #16, so one fix covers both call paths.

L-10

LOW

● FIXED

Fee manager accepts a looser swap floor than the vault

The fee manager lets harvest swaps fill up to 500 bps below TWAP, while the vault caps the same floor at 300 bps.

The `SmartLpFeeManager` constructor accepts `swapFloorBps_` up to 500 (`SmartLpFeeManager.sol:88`), and `setGuards` caps it at 300 (`SmartLpVault.sol:740`). Both add the pool fee and pass the result to `SmartLpLib.swapWithTwapFloor` . `harvest()` is permissionless and checks only this floor, so a sandwich attacker can take up to 200 extra bps of each harvest. Base-token fees of an external-target manager pass through two swaps and can lose about 400 extra bps.

IMPACT

Holders lose part of their harvested fees. Principal is not exposed.

RECOMMENDATION

Cap `swapFloorBps_` at 300 in the constructor.

● INFORMATIONAL SEVERITY · 5 FINDINGS

I-01

INFORMATIONAL

● FIXED

First deposit floor is skipped after a full exit

`SmartLpVault.sol#_mintShares()` uses the genesis branch only when `ts == 0 || tvBefore == 0`. After the last holder exits, `ts` is the 1000 dead shares and `tvBefore` is dust from rounding and the withdraw fee. Neither condition holds, so the next deposit is priced against 1000 wei of supply and `minFirstDepositQuote` is not checked.

[https://github.com/SB-Security/audit-2026-09-](https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L433-L446)

[SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L433-L446](https://github.com/SB-Security/audit-2026-09-SmartLpV2/blob/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L433-L446)

IMPACT

Deposits below the floor pass and shares get coarse (\$1 mints 27 shares). The dead shares absorb any donation.

RECOMMENDATION

```
- if (ts == 0 || tvBefore == 0) {  
+ if (ts ≤ DEAD_SHARES || tvBefore == 0) {
```

I-02

INFORMATIONAL

● FIXED

The Claimed event can emit address(0) as the fee receiver instead of the actual receiver

When `SmartLpFeeManager::claim` is called with `to = address(0)` the function defaults to sending the accrued fees to the `msg.sender`. In this case, the `Claimed` event is still incorrectly emitted using `to` for the fee receiver, which still contains `address(0)`

IMPACT

Indexers reading Claimed record the fees as sent to `address(0)`, not to the real receiver.

RECOMMENDATION

Check if `address(0)` was passed as `to`. A simple fix would be to set `if (to == address(0)) to = msg.sender;` before the transfer and use `to` in both places

I-03

INFORMATIONAL

● FIXED

Stale comments on `compound()`

Two comments describe `compound()` incorrectly:

- `SmartLpFeeManager.sol:136-137` says `compound()` can revert in "paused-adjacent states". No pause blocks `compound()`. It reverts only for other reasons, such as a failed observe or NFPM call.
- `SmartLpVault.sol:637` says `compound()` reinvests the collected fees. In harvest mode, the fees go to the fee manager, and `compound()` only redeploys idle balances.

IMPACT

Documentation mismatch

RECOMMENDATION

Update both comments to match the code.

I-04

INFORMATIONAL

● FIXED

Missing two step ownership transfer

`setRoles` (`SmartLpVault.sol:745-751`) replaces the owner, keeper, and guardian in one call, checking only that the new owner is not zero. This can lead to ownership being transferred to a wrong, unreachable address and permanently removes the ability to `setCap` , `setGuards` , `setRoles` , `sweep` , `setFeeManager` , and unpausing through `setDepositsPaused`

IMPACT

Loss of control of major functions of the protocol, as this can only happen due to an owner mistake; this is an informational severity issue.

RECOMMENDATION

We recommend splitting the owner out of `setRoles` into a two-step transfer, where the nominated owner must accept before taking control.

I-05

INFORMATIONAL

● FIXED

Three of four feed failure modes permanently block deposits and the keeper, against the documented guarantee

`checkTwapBand` documents an absolute guarantee about the price feed:

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLib.sol#L53-L56>

```
/// @notice Spot must sit within the (stale-widened) band of TWAP; when the Chainlink
///         feed is FRESH the pool-implied base price must also sit within maxVsFeedBps
///         of it. A stale or unreadable feed only widens the TWAP band - the feed is
///         never a liveness dependency (weekend equity gaps).
```

The implementation delivers that for one failure mode only. Two statements sit inside the `try` success body where a `catch` clause does not reach them - it only covers the external call itself:

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpLib.sol#L70-L81>

```
try IAggregatorV3Minimal(feed).latestRoundData() returns (
    uint80, int256 a, uint256, uint256 updatedAt, uint80
) {
    if (a > 0 && block.timestamp - updatedAt ≤ g.feedStaleAfter) {
        fresh = true;
        answer = a;
        fdec = IAggregatorV3Minimal(feed).decimals();
    }
} catch {}
if (!fresh) band = g.maxSpotVsTwapStaleBps;
```

`block.timestamp - updatedAt` is checked 0.8 arithmetic, so a timestamp ahead of the chain panics rather than reading as "not fresh". The `decimals()` call is a second external call with no protection at all.

Also `feed` is immutable, the constructor performs no code check on it and there is no setter:

<https://github.com/SB-Security/audit-2026-09-SmartLpV2/251f17e44d0a1c8c2c9ee866cfc84f5c4d9c3760/contracts/smart-lp/SmartLpVault.sol#L92>

So on a vault that ever reaches one of the three failing states, `deposit`, `zapDeposit`, `depositSingle`, `zapWithdraw`, `recenter` plus `trailDown` are dead permanently. `withdraw` never consults the feed so funds always remain recoverable. `compound` is unaffected too.

IMPACT

Deposits and the whole keeper surface stop working on that vault forever, with no admin path to repair it. Nothing is stealable and nothing is locked, because the in-kind exit is unguarded by design.

Being straight about how reachable this is: I checked it and it is not, today. Every one of the 27 distinct feeds used across the 80 feed-bearing live vaults has code, reports `decimals() = 8` plus returns an `updatedAt` at or behind the current block. A correctly deployed Chainlink aggregator proxy will not produce any of the three failing states, so this is a robustness and documentation defect rather than a live exposure. It is still worth fixing. The comment states a guarantee the code does not provide, the field is immutable, so a bad value at deployment or a feed that degrades later could never be corrected.

RECOMMENDATION

Move both statements out of the `try` body so every unusable feed lands on the same path the comment describes.

I-05 · CONTINUED

Read `decimals()` in its own `try` and compare the timestamp without subtracting:

```
try IAggregatorV3Minimal(feed).latestRoundData() returns (
    uint80, int256 a, uint256, uint256 updatedAt, uint80
) {
    if (a > 0 && updatedAt ≤ block.timestamp && block.timestamp - updatedAt ≤
        g.feedStaleAfter) {
        try IAggregatorV3Minimal(feed).decimals() returns (uint8 dd) {
            fresh = true;
            answer = a;
            fdec = dd;
        } catch {}
    }
} catch {}
```

A code-size check on `feed` in the constructor would also stop a mistyped address from producing a vault that can never take a deposit.

08 POST-AUDIT STATEMENT

Post-Audit Statement

The audit found two High and four Medium issues, and the fixes for them changed the code substantially. This section sets out what the fix review covered.

WHAT THE FIX REVIEW COVERED

Between the audited and the final commit, the three contracts in scope grew from about 1,400 to about 2,300 lines. Deposit pricing was redesigned and the fee manager's crediting was rewritten. Ask settlement, a depth-limited recenter and an early exit fee were added, and much of the vault's logic moved into the library to fit the contract size limit. We reviewed these changes to confirm that each reported finding is resolved. They were not audited with the depth of the original engagement, so this report does not cover issues the new code may introduce.

RECOMMENDED NEXT STEP

Given the severity of the findings and the size of the changes, we recommend a full review of the final code as a separate engagement. This is a normal step after a remediation of this size, not a judgment of the team, who fixed every reported issue within the remediation window.



AUDIT COMPLETE

Stay secure.

Thank you for choosing SBSecurity.

WEBSITE

sbsecurity.net

TELEGRAM

[@sbsecurity_audits](https://t.me/sbsecurity_audits)

REPORT

StonkBrokers Smart LP v2 ·
October 2026